DSA612S · Distributed Food Delivery Platform

Defense Walkthrough

Presenter: **Jason** — Order Service owner & system integration

What this document is

A speaking guide for today's defense. It walks the panel through the platform as a whole, then spends most of the time on **my part**: the **Order Service** (the central state machine every other service reacts to) and the **integration work** that made all seven services run together.

Blue boxes are things to **say**. Green boxes are tips for the **demo**. Orange boxes are **weaknesses to own** before the panel finds them.

Contents

1. Running order & timing
2. The system in one page
3. My part: Order Service — responsibilities & structure
4. The order state machine
5. Event choreography: how orders move on their own
6. Data design & consistency guarantees
7. REST API & validation
8. Testing strategy
9. My integration work across the team
10. Live demo script
11. Expected questions & answers
12. Limitations I'd fix next · code cheat-sheet

Service	Owner	Port	Store
Order Service	**Jason (me)**	**8081**	**MongoDB**
Customer Service	—	8082	MongoDB
Restaurant Service	Renate	8083	MongoDB
Payment Service	Albert	8084	PostgreSQL
Delivery Service	Penny	8085	MongoDB
Notification Service	Gerson	8086	MongoDB
Admin Service	Celine	8087	MongoDB
Compose, Kafka topics, Web UI, smoke test, docs	**Jason (integration)**	**3000**	**—**

1 · Running order & timing

Assumes roughly 15 minutes for my slot. Scale each block if the slot is shorter; never cut the demo.

Time	Block	Key message
0:00–1:00	Opening	"Seven Ballerina microservices, talking only through Kafka. I own the Order Service, which is the heart of the order lifecycle, and I did the integration that runs them all together."
1:00–3:00	System in one page	Database per service, events not calls, keyed by `orderId`.
3:00–5:00	**Order Service structure**	**Four modules, one-way dependencies, pure domain logic isolated for testing.**
5:00–7:00	**State machine**	**Seven statuses, explicit transitions, terminal states, why READY can't be cancelled.**
7:00–9:30	**Event choreography**	**Restaurant ACK and payment can arrive in either order; whichever is second advances the order.**
9:30–11:00	**Consistency**	**Optimistic concurrency, idempotent handlers, manual offset commit, `acks=all`.**
11:00–14:00	Live demo	Happy path CREATED → DELIVERED, then a rejected order that is cancelled and refunded.
14:00–15:00	Close	Testing summary, honest limitations (dual write → outbox), questions.

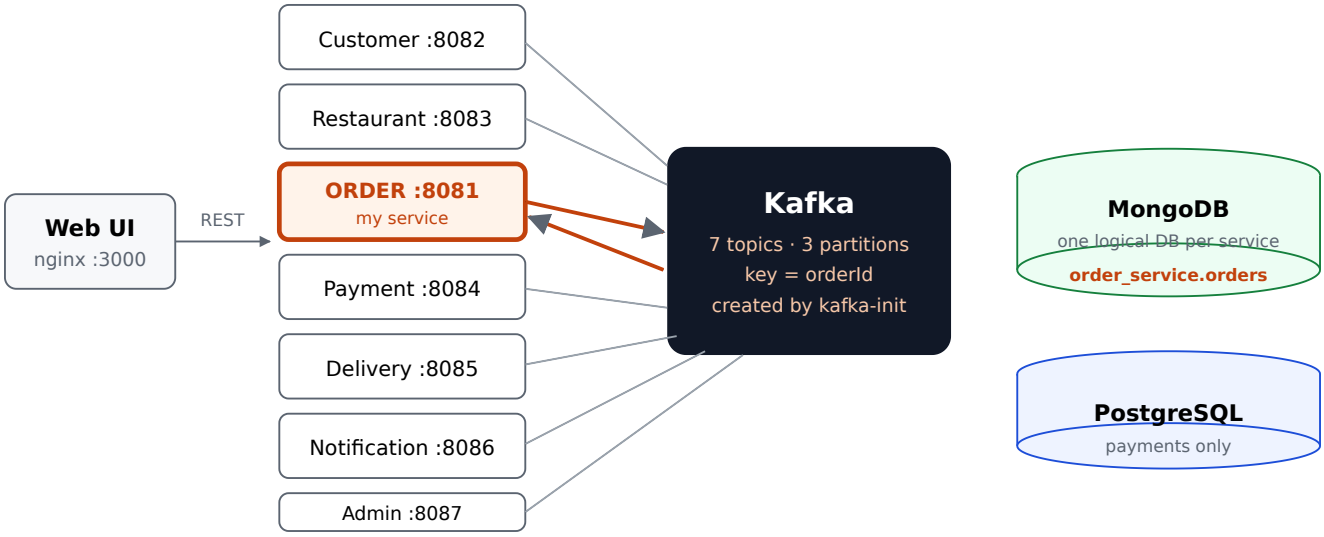
Opening line (memorise this)

> **Say:** "Our platform is a food-delivery system built as seven independent Ballerina microservices. They never call each other directly — every interaction is a Kafka event, and each service owns its own database. My part is the Order Service: it creates orders, owns the order's lifecycle, and moves each order forward automatically when the restaurant, payment and delivery services report back. I also wired the whole system together — Docker Compose, the Kafka topics, the web front-end and an end-to-end test — so I'll finish with a live demo of one order going all the way through."

Before you walk in

- `docker compose up -d --build` at least 5 minutes early; wait for all containers to be *healthy*.
- Open **http://localhost:3000 → System**, confirm all 7 health cards are green, click **Load demo data**.
- Run `node scripts/smoke-test.mjs` once as a dress rehearsal — every line should read `PASS`.
- Have `order-service/modules/messaging/consumer.bal` and `modules/lifecycle/types.bal` open in the editor.
- Keep a terminal ready with `docker compose logs -f order-service` — the "Auto-advanced order" log lines are great evidence.

2 · The system in one page



Four architectural rules (say them in this order)

1. **Events, not calls.** No service calls another service's API. Coupling is only on the shared Kafka contract in the root README.
2. **Database per service.** Each service owns its data; others keep their own *read models* built from events (CQRS style). If one service is down, the others still answer from their copy.
3. **Keyed by `orderId`.** All events for one order land on one partition, so they stay in order; different orders spread over 3 partitions for parallelism.
4. **At-least-once, idempotent consumers.** Kafka can redeliver, so every handler is safe to run twice (upserts, unique indexes, status guards).

Kafka contract (my two topics highlighted)

Topic	Producer	Consumers
`orders.created`	**Order**	**Customer, Restaurant, Payment, Notification, Admin**
`orders.status_updated`	**Order**	**Customer, Payment, Delivery, Notification, Admin**
`restaurant.status`	Restaurant	**Order**, Notification
`payments.completed`	Payment	**Order**, Delivery, Notification, Admin
`delivery.status_updated`	Delivery	**Order**
`delivery.assigned` / `delivery.completed`	Delivery	Notification, Admin

> **Say:** "Notice the Order Service is on both ends of the contract: it produces the two topics almost every service listens to, and it consumes the three topics that report progress back. That's why it is the centre of the choreography — but it still never calls anyone."

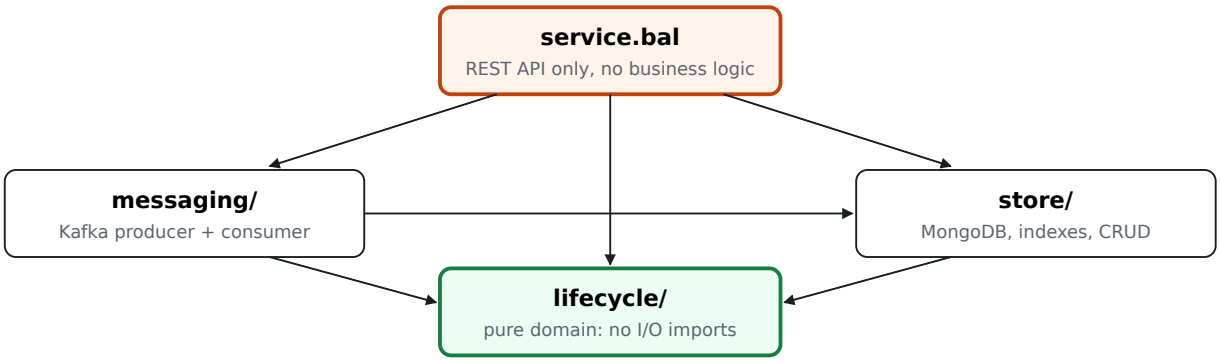
3 · My part: the Order Service

Ballerina 2201.13.5 | MongoDB | Kafka | port 8081

Responsibilities

- **Create orders** (`POST /orders`): validate, compute the total, save as `CREATED` , publish `orders.created` .
- **Own the lifecycle**: the only service allowed to change an order's status, and only along legal transitions.
- **Auto-advance** orders from `restaurant.status` , `payments.completed` and `delivery.status_updated` .
- **Announce every change** on `orders.status_updated` — which is what triggers refunds, delivery cancellation, notifications and admin stats elsewhere.
- **Serve queries** for the customer's "My orders" page and the restaurant's kitchen board.

Module layout — one responsibility per module



Arrows = "depends on". Nothing depends on `service.bal` ; `lifecycle` depends on nothing.

> **Say:** "I split the service into Ballerina submodules with one-way dependencies. The key one is `lifecycle` : it holds the order types, the transition rules, total calculation and validation, and it imports nothing — no HTTP, no Kafka, no Mongo. That means the most important rules in the system can be unit-tested in under a second with no infrastructure running, and both the REST API and the Kafka consumer use exactly the same rules."

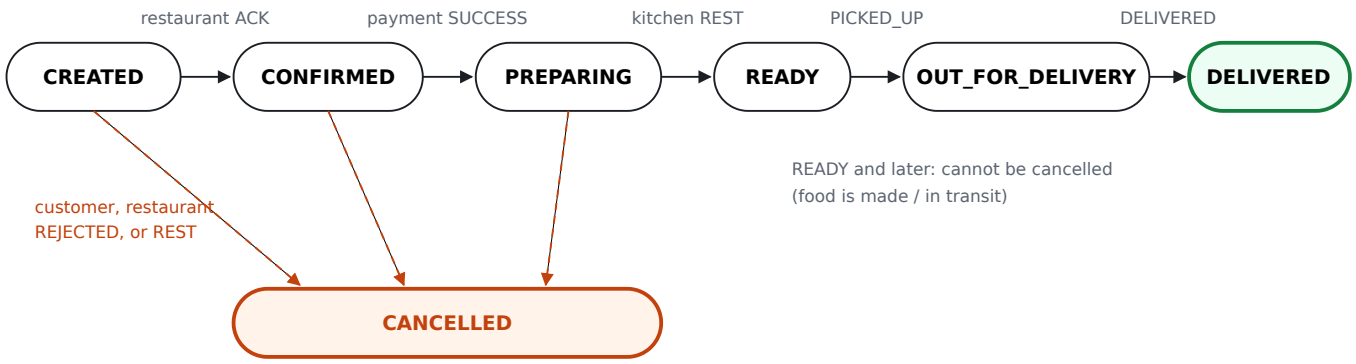
Why submodules and not just folders?

Ballerina only compiles `.bal` files sitting directly in a module directory — arbitrary subfolders are silently ignored. So the "folders" are real modules (`modules/<name>/`), which also gives us enforced boundaries: `store` cannot accidentally import `messaging` .

One non-obvious detail worth mentioning

`service.bal` calls `messaging:consumedTopics()` for the health endpoint. Besides reporting what we consume, it guarantees the module is imported — in Ballerina an unimported module's Kafka listener never starts.

4 · The order state machine



From	Allowed next	Who / what triggers it
CREATED	CONFIRMED, CANCELLED	`restaurant.status` ACKNOWLEDGED / REJECTED, or customer cancel
CONFIRMED	PREPARING, CANCELLED	`payments.completed` SUCCESS (or the ACK if payment came first)
PREPARING	READY, CANCELLED	Restaurant clicks **Mark ready** → `PUT /orders/{id}/status`
READY	OUT_FOR_DELIVERY	`delivery.status_updated` PICKED_UP
OUT_FOR_DELIVERY	DELIVERED	`delivery.status_updated` DELIVERED
DELIVERED / CANCELLED	— (terminal)	Every further request gets **409 Conflict**

How it is enforced

One function, `isValidTransition(current, next)` in `modules/lifecycle/types.bal:55`, is a `match` on the current status. It is the single source of truth: the REST endpoint checks it before writing, and the Kafka consumer only ever asks for legal steps.

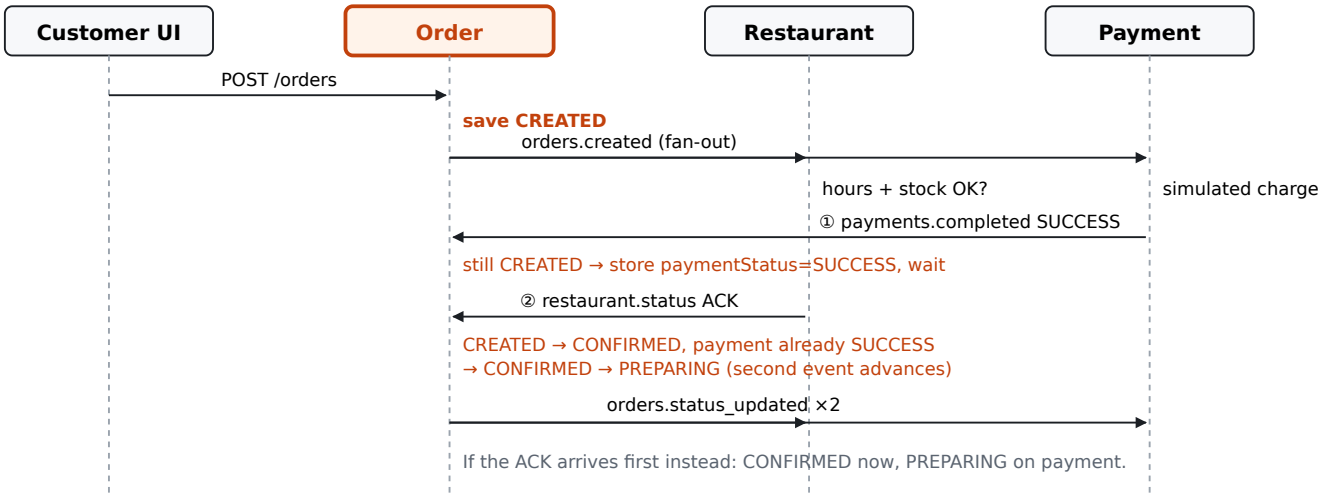
> **Say:** "The state machine is deliberately strict. You can't skip steps, you can't go backwards, and terminal states stay terminal. Cancellation is only allowed up to PREPARING — once the food is ready we can't un-cook it, and once it's with the driver the delivery service also refuses to cancel. Every illegal request gets a 409, not a silent overwrite."

Catch-up behaviour

If the driver marks an order DELIVERED without a separate pick-up (order still READY), the consumer walks it READY → OUT_FOR_DELIVERY → DELIVERED using two *legal* transitions, so the history and the published events stay correct (`consumer.bal:154`).

5 · Event choreography: how orders move on their own

The hardest design problem in my service: **the restaurant's acknowledgement and the payment result are produced by two different services on two different topics, so they can arrive in either order.**



The rule: "whichever event arrives second advances the order"

- **Payment handler** (`consumer.bal:67`) always stores `paymentStatus` on the order. If the order is already CONFIRMED and payment succeeded → advance to PREPARING. If still CREATED → do nothing; the ACK will finish the job.
- **Restaurant handler** (`consumer.bal:92`) moves CREATED → CONFIRMED, then checks the stored `paymentStatus` ; if it is already SUCCESS it continues straight to PREPARING.
- A **FAILED** payment is recorded but the order stays put — the UI shows **Retry payment**, and a later SUCCESS advances it normally.

Rejection path (cancel + refund without any direct call)

```
restaurant.status REJECTED (reason: "closed" / "out of stock")
   └─ Order: store cancellationReason, CREATED|CONFIRMED → CANCELLED
        └─ orders.status_updated CANCELLED
             ├─ Payment:  refund any SUCCESS payment, remember order as cancelled
             ├─ Delivery: cancel delivery if not yet picked up
             └─ Notification / Admin / Customer: update their read models
```

> **Say:** "My service doesn't know refunds exist. It just publishes 'this order is now CANCELLED', and the payment and delivery services each decide what that means for them. That's choreography rather than orchestration — adding a new reaction to cancellation needs zero changes in my code."

6 · Data design & consistency guarantees

The `orders` document

```
{
  "orderId": "uuid",          // unique index
  "customerId": "uuid",
  "restaurantId": "uuid",
  "items": [{ "itemId", "name",
              "quantity", "unitPrice" }],
  "totalAmount": 90.00,       // decimal
  "status": "PREPARING",
  "paymentStatus": "SUCCESS", // optional
  "cancellationReason": "...",// optional
  "createdAt": "ISO-8601",
  "updatedAt": "ISO-8601"
}
```

Design decisions

- **Items embedded as a snapshot** of name and price at order time — if the restaurant changes its prices later, past orders don't change, and reading an order needs no join.
- **`decimal`, not float**, for money.
- **IDs, not foreign keys** — `customerId` / `restaurantId` live in other services' databases.
- **Optional fields** (`paymentStatus?`) so documents written before a field existed still deserialise.

Indexes (created at startup, `store.bal:31`)

- `{orderId:1}` unique — the platform-wide key
- `{customerId:1, createdAt:-1}` — "My orders", newest first
- `{restaurantId:1, createdAt:-1}` — kitchen board

Five guarantees and how each one is achieved

Guarantee	Mechanism
No lost updates under concurrency	**Optimistic concurrency.** `applyStatusTransition` updates with filter `{orderId, status: previousStatus}` . If a REST call and a Kafka event race, only one matches; the loser gets `matchedCount == 0` → an error / 409 instead of overwriting (`store.bal:77`).
Per-order ordering	Every event keyed by `orderId` → same partition → processed in order.
Redelivery is harmless (idempotency)	Each handler checks the current status first: an ACK for an order that is no longer CREATED, or a pickup for an order that isn't READY, is logged and skipped.
No silent message loss on consume	`autoCommit: false` ; offsets committed only after the batch has been handled (`consumer.bal:22–42`).
Durable publish	Producer uses `acks: "all"` with `retryCount: 3` (`producer.bal:13`).

> **Say:** "Kafka gives us at-least-once delivery, so I designed every handler on the assumption that it will see the same event twice. The status check at the top of each handler plus the optimistic update in Mongo means replaying the whole topic from the beginning would leave every order exactly where it is."

> **Own it:** a record that fails to parse is logged and its offset is still committed, so it is effectively dropped. That keeps one bad message from blocking the partition, but in production I'd route it to a **dead-letter topic** instead.

7 · REST API & validation

Method	Path	Behaviour	Errors
POST	`/orders`	Validate → compute total → save CREATED → publish `orders.created`	400 invalid, 500 DB/Kafka
GET	`/orders`	Newest first; filters `?customerId=` `?restaurantId=` `?status=`	500
GET	`/orders/{id}`	One order	404
PUT	`/orders/{id}/status`	Check `isValidTransition` → optimistic update → publish `orders.status_updated`	404, 409 illegal or concurrent
GET	`/health`	`{status, consumedTopics}`	—

`/health` is its own service base path so it can never be mistaken for an `orderId`.

Validation rules (`validateCreateOrder`, `types.bal:89`)

- `customerId` and `restaurantId` must be non-blank.
- At least one item; each item needs an `itemId` and a `name`.
- Quantity between **1 and 50** (`MAX_ITEM_QUANTITY`).
- Unit price strictly greater than zero.
- The **total is computed on the server** (`computeOrderTotal`) — the client never sends an amount, so it can't under-pay.

Ballerina also rejects malformed JSON or unknown status values automatically, because request bodies bind to closed records and the `OrderStatus` enum.

What the Order Service deliberately does *not* validate

It does not check that the restaurant is open or the dish is in stock — that data belongs to the Restaurant Service. Instead the order is accepted as CREATED and the restaurant answers asynchronously with ACKNOWLEDGED or REJECTED. This keeps services decoupled: placing an order works even if the restaurant service is briefly down; the order simply waits in CREATED until it recovers.

> **Say:** "Validation is split by ownership. I validate what my service knows — shape, quantities, prices. Business rules about the restaurant are validated by the restaurant, and come back as an event. That's the price of not having synchronous calls, and it's also what makes the system resilient."

Sample request

```
POST /orders
{ "customerId": "c-1", "restaurantId": "r-1",
  "items": [{ "itemId": "i-1", "name": "Kapana beef strips", "quantity": 2, "unitPrice": 45.0 }] }
→ 200 { "orderId": "…", "totalAmount": 90.0, "status": "CREATED", … }
```

8 · Testing strategy

Three tiers, from fastest to most realistic.

Tier	What	Needs	Command
Unit	**All 49 from→to combinations of `isValidTransition` (7×7 matrix via a data provider), `computeOrderTotal`, and `validateCreateOrder` accept/reject cases**	**Nothing**	`bal test --tests "order_service.lifecycle:*"`
Integration	**7 tests against the real HTTP API + Mongo + Kafka: create, empty items → 400, unknown → 404, valid transitions, invalid → 409, list**	**Kafka, Mongo**	`bal test --groups integration …`
End-to-end	**`scripts/smoke-test.mjs`: one order CREATED → DELIVERED across all 7 services, a closed restaurant rejecting → CANCELLED + refunded, notifications, admin stats, validation**	**Full stack**	`node scripts/smoke-test.mjs`

> **Say:** "The expected values in the transition matrix were written from the state diagram, not copied from the code — so the test actually checks the logic instead of restating it. Testing all 49 pairs means every 'not allowed' case is proven, not just the happy path."

Why the domain module made this possible

The `store` and `messaging` modules connect to Mongo and Kafka the moment they initialise, so anything importing them needs infrastructure. Putting the rules in `lifecycle` with zero I/O imports means the most important tests run anywhere, in well under a second.

9 · My integration work across the team

Beyond my own service, the commit history shows I took on making the seven services work as **one system**:

Area	What I did
Docker Compose	Every service in one `docker-compose.yml`; healthchecks and restart policies; fixed image builds (Java 21 runtime for the Mongo/Kafka connectors).
Kafka topics	One-shot `kafka-init` container creates all 7 topics with 3 partitions *before* any service subscribes; every service waits on it.
Kafka contract	Wrote the single-source-of-truth topic table in the root README; consumers use **open records** so extra fields never break anyone.
Cross-service fixes	Payment double-charging and event contract; restaurant menu ownership checks and opening hours/stock; delivery cancellation before payment; notification storage; admin Kafka listener crash on startup.
Web front-end	Dependency-free UI (customer, restaurant, driver, admin, system tabs) behind an nginx proxy at `/api/<service>/`.
Verification & docs	End-to-end smoke test, `DATABASE.md`, architecture diagram.

> **Tip:** credit teammates by name when the panel asks about their services ("Albert's payment service uses a partial unique index…"), then bring it back to how it plugs into the order lifecycle.

10 · Live demo script

About 3 minutes. Narrate the **Order Service status** at every step — that's your part.

A · Happy path

1. **System tab** — show 7 green health cards and the topics each consumes. Point at the Order card: `restaurant.status, payments.completed, delivery.status_updated`.
2. **Customer tab** — pick a dish, place the order. *"My service saved it as CREATED and published orders.created to five consumers."*
3. Watch the tracker go **CREATED → CONFIRMED → PREPARING** on its own. *"Restaurant acknowledged, payment succeeded — whichever arrived second pushed it to PREPARING."* Show the `Auto-advanced order` lines in `docker compose logs order-service`.
4. **Restaurant tab** — click **Mark ready**. *"That's the one manual step: a REST call to PUT /orders/{id}/status, checked against the state machine."*
5. **Driver tab** — the delivery was already assigned. **Confirm pickup** → order becomes OUT_FOR_DELIVERY; **Mark delivered** → DELIVERED.
6. **Admin tab** — stats updated; **Customer inbox** shows notifications. *"None of those services ever called mine — they all read my events."*

B · Rejection & refund

7. **Restaurant tab → Pause orders**. Place another order as the customer.
8. Order goes to **CANCELLED** with the reason shown. Payment shows **REFUNDED**. *"Restaurant sent REJECTED; I cancelled and published it; payment refunded on its own."*

C · Prove the state machine (optional, if time)

```
curl -X PUT localhost:8081/orders/<delivered-id>/status \
     -H 'Content-Type: application/json' -d '{"status":"CANCELLED"}'
→ 409 {"message":"Invalid transition from DELIVERED to CANCELLED"}
```

If the demo stalls: about 15% of payments fail on purpose. If an order sits in CONFIRMED, that's a *failed payment* — click **Retry payment** and turn it into a talking point: "the order waits safely; the retry's SUCCESS event advances it". If something is truly broken, fall back to `node scripts/smoke-test.mjs` output.

Ports: UI 3000 · order 8081 · Kafka from host `localhost:29092` · Mongo `27017`. Inspect data: `docker compose exec mongo mongosh → use order_service → db.orders.find().sort({createdAt:-1}).limit(1)`.

11 · Expected questions & answers

About the Order Service (most likely)

Q What happens if the payment arrives before the restaurant acknowledges?
A The payment handler stores `paymentStatus = SUCCESS` but leaves the order in CREATED. When the ACK arrives, the restaurant handler moves it to CONFIRMED and, seeing the stored SUCCESS, straight on to PREPARING. Either order of arrival ends in the same state.

Q What if a REST status update and a Kafka event change the same order at the same moment?
A Both use `applyStatusTransition`, which updates with filter `{orderId, status: expectedPrevious}`. Mongo's single-document update is atomic, so only one matches. The REST caller gets 409 "changed concurrently, please retry"; the consumer logs the error. No write is lost or overwritten.

Q What if Kafka delivers the same event twice?
A Each handler checks the current status first. A second ACK finds the order no longer CREATED and is skipped; a second pickup finds it no longer READY. The optimistic filter is a second safety net. So handlers are idempotent.

Q Why not have the Order Service call Payment and Restaurant directly (orchestration)?
A Direct calls would couple availability — if Payment is down, nobody can order. With events, I publish once and each service reacts when it can; new reactions (notifications, admin stats) were added with no change to my code. The trade-off is eventual consistency and harder tracing, which we accept and mitigate with the health page and logs.

Q Why MongoDB for orders?
A An order is a self-contained aggregate — header plus embedded items — always read and written as one unit. A document fits naturally, single-document updates are atomic (which my optimistic concurrency relies on), and there are no cross-entity joins. Payments chose Postgres because money needs constraints and ACID across rows.

Q Why are items copied into the order instead of referenced?
A An order is a historical record. If the restaurant changes a price or deletes a dish, past orders must still show what the customer paid. It also avoids a cross-service lookup.

Q Why can't a READY order be cancelled?
A The food has been made and the driver may be on the way; cancelling would waste food and conflict with the delivery service, which also refuses to cancel after pickup. It's a business rule encoded in `isValidTransition`.

Q How do you guarantee events for one order are processed in order?
A All topics are keyed by `orderId`, so Kafka puts every event for that order on the same partition, and a partition is consumed in order. Across topics there's no ordering guarantee — which is exactly why I handle ACK/payment in either order.

Q What if your service crashes after saving the order but before publishing?
A That's the dual-write problem: the order exists in Mongo but no one hears about it, and the customer gets a 500. The proper fix is a **transactional outbox** — write the event into an outbox collection in the same write as the order, and have a relay publish it. It's documented as a known limitation.

Q What does an unknown `orderId` in an incoming event mean?
A Normally impossible, because I only publish `orders.created` after the save succeeds. If it happens (e.g. test data) it's logged as a warning and skipped, so it can't crash the consumer.

Q How would you scale the Order Service?
A Run more instances in the same consumer group — Kafka spreads the 3 partitions across them, and per-order ordering is preserved because one order is always on one partition. The REST side is stateless, and the optimistic concurrency keeps instances from clobbering each other. Beyond 3 consumers we'd add partitions.

About the system as a whole

Q How does a cancelled order get refunded?
A I publish `orders.status_updated` with CANCELLED. The Payment Service flips the SUCCESS row to REFUNDED and publishes `payments.completed` REFUNDED; it also records the order in `cancelled_orders` so a late charge can never go through.

Q Can a customer be charged twice?
A No — Payment has a partial unique index allowing at most one SUCCESS row per order, enforced by Postgres even if two attempts race past the code check.

Q Can a driver get two deliveries at once?
A No — Delivery claims a driver with filter `{driverId, status: AVAILABLE}`, the same optimistic pattern I use for orders. Only one claim can match.

Q Can stock go negative?
A No — Restaurant decrements with `{stockCount: {$gte: quantity}}` plus `$inc` in one atomic update.

Q Why one MongoDB server if it's "database per service"?
A It's one server for convenience in Compose, but each service has its own logical database and never touches another's. Moving one to its own server only changes `Config.toml`.

Q How do services stay compatible as events evolve?
A One documented contract in the README, and consumers use **open records**, so a producer can add fields without breaking anyone. Removing or renaming a field would need a versioned topic.

Q How do you know the whole thing works?
A The smoke test drives a real order through all seven services through the same nginx proxy the UI uses, including the rejection/refund path, and exits non-zero on any failure.

12 · Limitations I'd fix next

> Raise these yourself — it shows you understand the design rather than defending it blindly.

Limitation	Impact	Fix
Dual write (Mongo then Kafka)	Crash between the two leaves DB and events out of sync	Transactional outbox + relay (or Debezium CDC)
Bad records are logged and skipped	A malformed event is effectively lost	Dead-letter topic + alerting
No authentication / authorisation	Anyone can call any API, e.g. advance any order	JWT at the gateway; roles (customer / restaurant / driver)
Single Kafka broker, replication 1	Broker loss = outage	3 brokers, replication factor 3, `min.insync.replicas=2`
No distributed tracing	Hard to follow one order across 7 services	Propagate a correlation ID; Ballerina observability is already enabled in `Ballerina.toml`
No order timeout	An order whose restaurant never replies stays CREATED	Scheduled job cancels stale CREATED orders

Code cheat-sheet

Open these if the panel says "show me".

Concept	Where
State machine	`order-service/modules/lifecycle/types.bal:55`
Request validation	`order-service/modules/lifecycle/types.bal:89`
Create order endpoint	`order-service/service.bal:22`
Status update endpoint	`order-service/service.bal:93`
Kafka listener, manual commit	`order-service/modules/messaging/consumer.bal:22`
Payment handler (either-order logic)	`order-service/modules/messaging/consumer.bal:67`
Restaurant ACK / REJECT handler	`order-service/modules/messaging/consumer.bal:92`
Delivery handler (catch-up)	`order-service/modules/messaging/consumer.bal:133`
Producer, `acks=all`, keyed by orderId	`order-service/modules/messaging/producer.bal:13`
Indexes	`order-service/modules/store/store.bal:31`
Optimistic concurrency	`order-service/modules/store/store.bal:77`
49-case transition test	`order-service/modules/lifecycle/tests/lifecycle_test.bal`

> **Closing line:** "To sum up: the Order Service is the single owner of the order lifecycle, it enforces a strict state machine, and it moves orders forward purely from events — safely under redelivery, out-of-order arrival and concurrent updates. Thank you — I'm happy to take questions."